COURS AVANCÉ · DÉVELOPPEMENT FULL-STACK

Next.js, Clean Architecture et Monorepo

Construire une application maintenable : App Router, couches métier, Turborepo, tests et CI/CD

Next.js App Router | TypeScript | Turborepo | pnpm | Prisma | Vitest

Med Amine Ben Rhouma · 2026



INTRODUCTION

Objectifs du cours

Maîtriser l'App Router

Server Components, rendu, data fetching, Server Actions.

Structurer en couches

Domain, application, infrastructure, presentation.

Protéger le métier

Règle de dépendance, ports et adaptateurs, injection.

Industrialiser le monorepo

pnpm workspaces, Turborepo, packages internes, Prisma.

Tester chaque couche

Vitest, fakes, tests d'intégration, Playwright.

Livrer en continu

CI GitHub Actions, cache distant, Vercel ou Docker.

INTRODUCTION

Programme

PARTIE 01
Next.js App Router

- Routing et layouts
- Server / Client Components
- Rendu et cache
- Server Actions
- Route Handlers

PARTIE 02
Clean Architecture

- La règle de dépendance
- Entités et use cases
- Ports et adaptateurs
- Validation et erreurs
- Injection de dépendances

PARTIE 03
Monorepo

- pnpm workspaces
- Turborepo et cache
- Packages internes
- Prisma partagé
- Contraintes de couches

PARTIE 04
Qualité et livraison

- Stratégie de tests
- CI GitHub Actions
- Vercel ou Docker
- Projet TaskFlow
- Pièges courants

INTRODUCTION

Prérequis

- React : composants, props, hooks
- TypeScript : interfaces, génériques, types
- Node.js 20+ et pnpm installés
- Notions de SQL et de schéma relationnel
- Git et pull requests
- Avoir déjà créé une application React

> Le cours React et Hooks est le prérequis direct de cette séance.

ce-qu-il-faut-savoir.ts

```ts
// TypeScript : interfaces, génériques, types utilitaires
interface Repository<T> { save(e: T): Promise<void>; }

// async / await et Promise
const data = await repo.findAll();

// Modules ESM et imports nommés
import { createTask } from '@acme/application';

// React : composants, props, hooks
export function Liste({ items }: { items: Task[] }) {
  return <ul>{/* … */}</ul>;
}

// Bases SQL et notions de schéma relationnel
// Git : branches, pull requests
```

01

Next.js : l’App Router

Routing, Server Components, stratégies de rendu, cache, Server Actions et Route Handlers

PARTIE 1 · DÉCOUVERTE

Qu’est-ce que Next.js ?

Un framework full-stack bâti sur React : routing, rendu serveur, API et build, dans un seul projet.

Vercel

Développé par Vercel, avec l’équipe React.

RSC

Les React Server Components sont le modèle par défaut.

Full-stack

UI, accès aux données et endpoints dans la même base de code.

PARTIE 1 · DÉCOUVERTE

Ce que le framework apporte

Routing par fichiers

L'arborescence de app/ définit les URL, les layouts et les états de chargement.

Rendu serveur

HTML généré côté serveur : statique, dynamique, incrémental ou en streaming.

Accès direct aux données

Un Server Component peut interroger la base sans passer par une API HTTP.

Server Actions

Des mutations appelées depuis le client, exécutées sur le serveur.

Build intégré

Bundling, découpage du code, optimisation des images et des polices.

Déploiement

Sortie standalone pour Docker, ou déploiement géré sur Vercel.

Créer une application Next.js

Terminal

```
# Créer l application (TypeScript + App Router)
pnpm create next-app@latest taskflow

# Options proposées
# TypeScript ..... Yes
# ESLint / Biome ..... Yes
# Tailwind CSS ..... Yes
# App Router ..... Yes
# Turbopack ..... Yes

cd taskflow
pnpm dev   # http://localhost:3000
```

⚠ Pages Router ?

L'ancien dossier pages/ reste supporté, mais tout le cours utilise app/.

⚙ Convention

Un seul projet aujourd'hui ; la partie 3 le transformera en monorepo.

⚡ Turbopack

Le bundler de développement de Next.js : démarrage et rebuild rapides.

PARTIE 1 · APP ROUTER

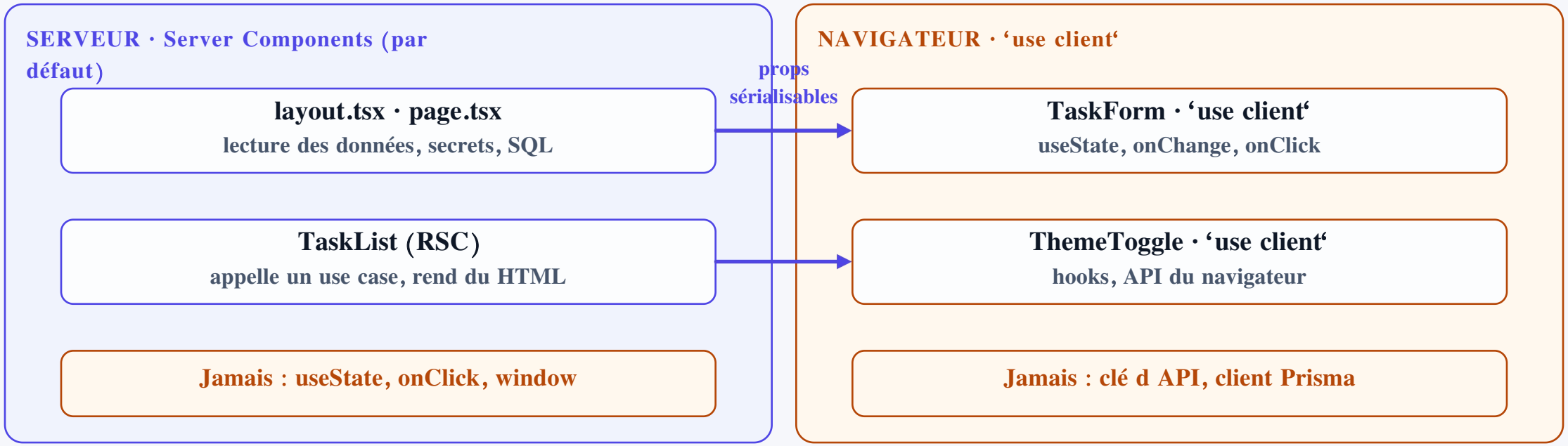
Le routing par fichiers

```
app/
├── layout.tsx          → enveloppe toutes les pages
├── page.tsx            → /
├── loading.tsx         → état de chargement
├── error.tsx           → erreurs du segment
├── tasks/
│   ├── page.tsx        → /tasks
│   ├── [id]/page.tsx   → /tasks/42
│   └── (admin)/
│       └── stats/page.tsx  → /tasks/stats
└── api/
    └── tasks/route.ts  → GET/POST /api/tasks
```

Fichier	Rôle
page.tsx	Rend une URL
layout.tsx	UI partagée, conservée
loading.tsx	Suspense automatique
error.tsx	Error boundary du segment
route.ts	Endpoint HTTP
[id]	Segment dynamique
(groupe)	Organisation sans URL

Seuls page.tsx et route.ts créent une URL : tout autre fichier peut vivre dans app/ sans être exposé.

Server et Client Components



Un Server Component peut importer un Client Component et lui passer des props sérialisables. L’inverse est impossible : la frontière ne se traverse que dans un sens.

Choisir le type de composant

Besoin	Server Component	Client Component
Lire la base de données	Oui	Non
Utiliser une clé secrète	Oui	Non
useState, useEffect	Non	Oui
onClick, onChange	Non	Oui
Accès à window ou localStorage	Non	Oui
Réduire le JavaScript envoyé	Oui	Non
Appeler un use case métier	Oui	via Server Action

Règle pratique : commencer serveur, descendre « use client » vers les feuilles interactives.

PARTIE 1 · RENDU

Les stratégies de rendu

Stratégie	Quand le HTML est produit	Cas d'usage
Statique (SSG)	Au build	Pages marketing, documentation
Dynamique (SSR)	À chaque requête	Tableau de bord, données par utilisateur
ISR	Au build, puis revalidé	Catalogue, blog, données semi-fraîches
Streaming	Par morceaux, avec Suspense	Pages lentes : afficher le reste tout de suite
Client	Dans le navigateur	Vues très interactives, données live

app/tasks/page.tsx

```
export const revalidate = 60; // ISR : regénère après 60 s

export default async function Page() {
  const tasks = await listTasks(); // exécuté côté serveur
  return <TaskList tasks={tasks} />;
}
```

⚠ Lire les cookies ou les en-têtes rend la route dynamique

Un loading.tsx ou un <Suspense> suffit à streamer la page

generateStaticParams pré-rend les routes dynamiques

PARTIE 1 · DONNÉES

Récupérer et invalider les données

lecture des données

```
// 1. Directement dans un Server Component
const tasks = await prisma.task.findMany(); // à isoler ! (partie 2)

// 2. Via fetch, avec contrôle du cache
const res = await fetch(`${API}/tasks`, {
  cache: 'force-cache',        // mise en cache explicite
  next: { revalidate: 60, tags: ['tasks'] },
});

// 3. Invalidation après une mutation
revalidateTag('tasks');
revalidatePath('/tasks');
```

Depuis Next.js 15

fetch n'est plus mis en cache par défaut : le cache devient explicite.

Tags

Marquer les lectures, puis invalider par tag après une écriture.

En parallèle

Promise.all pour éviter les requêtes en cascade.

⚠ Appeler Prisma directement dans un composant « marche » — c'est justement le couplage que la partie 2 va supprimer.

PARTIE 1 · MUTATIONS

Les Server Actions

app/tasks/actions.ts

```ts
'use server';

export async function createTaskAction(formData: FormData) {
  const titre = String(formData.get('titre'));
  await createTask({ titre });      // use case (partie 2)
  revalidatePath('/tasks');
}
```

app/tasks/TaskForm.tsx

```tsx
'use client';

export function TaskForm() {
  return (
    <form action={createTaskAction}>
      <input name='titre' />
      <button>Ajouter</button>
    </form>
  );
}
```

1 Déclarer

« use server » marque une fonction exécutable sur le serveur.

2 Appeler

Depuis un formulaire (action) ou un gestionnaire d'événement.

3 Invalider

revalidatePath ou revalidateTag rafraîchit l'affichage.

Route Handlers : quand les utiliser

```ts
// app/api/tasks/route.ts
import { NextResponse } from 'next/server';

export async function GET() {
  const tasks = await listTasks();        // use case
  return NextResponse.json(tasks);
}

export async function POST(req: Request) {
  const body = await req.json();
  const task = await createTask(body);    // use case
  return NextResponse.json(task, { status: 201 });
}
```

Oui

Client externe, application mobile, webhook, intégration tierce.

Inutile

Pour vos propres pages : un Server Component ou une action suffit.

Même règle

Un handler est une couche de transport : il appelle un use case, rien de plus.

PARTIE 1 · TRANSVERSE

Middleware et garde d’accès

```ts
// middleware.ts (racine du projet)
import { NextResponse, type NextRequest } from 'next/server';

export function middleware(req: NextRequest) {
  const session = req.cookies.get('session');
  if (!session) {
    return NextResponse.redirect(new URL('/login', req.url));
  }
  return NextResponse.next();
}

export const config = { matcher: ['/dashboard/:path*'] };
```

Avant la route

Exécuté sur chaque requête correspondant au matcher.

Pas de métier

Pas de Prisma, pas de calcul lourd : redirection et en-têtes.

Défense en profondeur

Vérifier aussi les droits dans le use case appelé.

Le middleware protège les URL ; l’autorisation métier reste dans la couche application.

02

Clean Architecture

Protéger les règles métier des détails techniques : couches, ports, adaptateurs et injection de dépendances

PARTIE 2 · LE PROBLÈME

Quand tout vit dans la page

app/tasks/page.tsx — anti-pattern

```
export default async function Page() {
  const tasks = await prisma.task.findMany({   // base de données
    where: { archived: false },
    orderBy: { createdAt: 'desc' },
  });
  // règle métier perdue dans la vue
  const urgentes = tasks.filter(
    (t) => t.dueDate && t.dueDate < new Date() && !t.done
  );
  await fetch('https://api.mailer.io/notify', { /* ... */ });
  return <TaskList tasks={urgentes} />;
}
```

⚠ Impossible à tester

Tester la règle « urgente » exige une base de données et un serveur.

⚠ Règle dupliquée

La même règle sera réécrite dans l'API, le cron, le mobile.

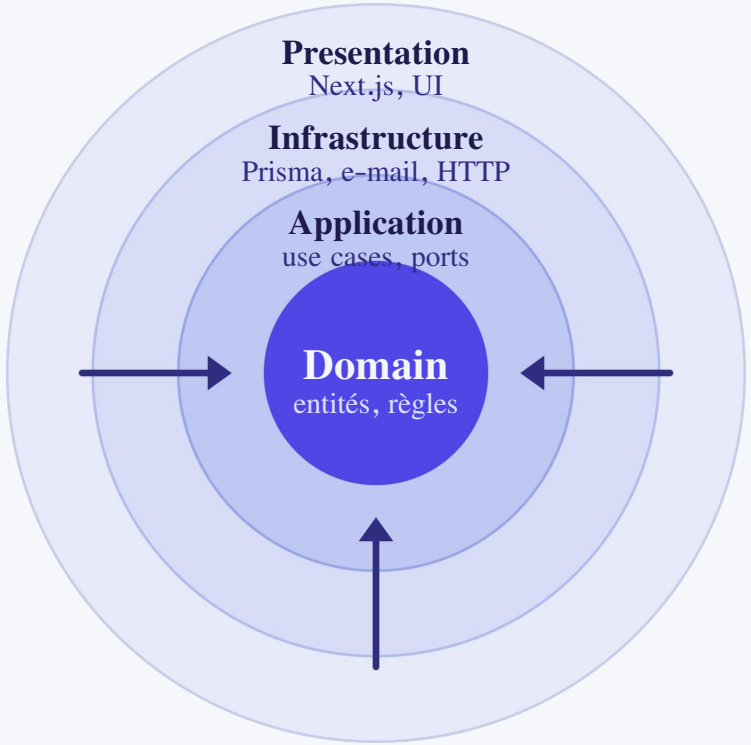
⚠ Couplage total

Changer d'ORM ou de framework impose de réécrire les vues.

⚠ Métier invisible

Aucun fichier ne répond à : « que fait cette application ? »

Les quatre cercles



Domain
Entités, objets valeur, règles métier pures.
Dépend de : rien

Application
Use cases : orchestrent le domaine, déclarent les ports.
Dépend de : domain

Infrastructure
Adaptateurs : Prisma, e-mail, stockage, API externes.
Dépend de : application, domain

Presentation
App Router, Server Components, actions, handlers.
Dépend de : application

PARTIE 2 · PRINCIPES

La règle de dépendance

Une couche interne ne connaît jamais le nom d'une couche externe.

Violation — le domaine importe la technique

```
import { prisma } from '@acme/db'; // interdit
import { headers } from 'next/headers'; // interdit

export class Task {
  async save() {
    await prisma.task.create({ /* … */ });
  }
}
```

Correct — le domaine reste pur

```
export class Task {
  private constructor(
    readonly id: TaskId,
    private title: string,
    private done = false,
  ) {}

  complete() { this.done = true; } // règle pure
}
```

- Ni Prisma, ni Next.js, ni React dans domain et application
- Les dépendances s'inversent par des interfaces
- La partie 3 fera respecter la règle par un lint

PARTIE 2 · STRUCTURE

Les couches dans le projet

Couche	Contenu	Package (partie 3)
Domain	Entités, objets valeur, erreurs métier, règles	packages/domain
Application	Use cases, ports, DTO, validation d'entrée	packages/application
Infrastructure	Repositories Prisma, e-mail, stockage, mappers	packages/infra
Presentation	app/, Server Components, actions, handlers, UI	apps/web
Composition	Assemblage : quelle implémentation pour quel port	apps/web/src/container

Le test du nom

Un dossier doit parler métier (tasks, billing), pas technique (utils, helpers).

Le test de l'import

Ouvrir un fichier du domaine : s'il importe un outil, l'architecture fuit.

Le test du remplacement

Changer d'ORM ne doit toucher qu'une seule couche.

PARTIE 2 · DOMAIN

L’entité porte les règles

packages/domain/src/task/task.ts

```ts
export class Task {
  private constructor(
    readonly id: TaskId,
    private props: TaskProps,
  ) {}

  static create(title: string, dueDate?: Date) {
    if (title.trim().length < 3) {
      throw new TitreTropCourt(title);
    }
    return new Task(TaskId.next(), {
      title, done: false, dueDate,
    });
```

… suite : les comportements métier

```ts
  complete() {
    if (this.props.done) {
      throw new TacheDejaTerminee(this.id);
    }
    this.props.done = true;
  }

  estUrgente(now = new Date()) {
    const { done, dueDate } = this.props;
    return !done && !!dueDate && dueDate < now;
  }
```

Toujours valide

Constructeur privé : on passe par create(), qui vérifie les invariants.

Pas de setter

complete() exprime une intention ; done = true ne dit rien.

Testable seul

Aucune dépendance : le test unitaire dure quelques millisecondes.

PARTIE 2 · DOMAIN

Objets valeur et erreurs métier

packages/domain/src/task/task-id.ts

```ts
export class TaskId {
  private constructor(readonly value: string) {}

  static next() {
    return new TaskId(crypto.randomUUID());
  }

  static from(value: string) {
    if (value.length !== 36) throw new TaskIdInvalide(value);
    return new TaskId(value);
  }

  equals(autre: TaskId) {
    return this.value === autre.value;
  }
```

erreurs du domaine

```ts
export class TitreTropCourt
  extends Error {
  readonly code = 'TITRE_TROP_COURT';
  constructor(titre: string) {
    super(`Titre trop court :  {titre}`);
  }
}
```

Pourquoi ?

Un string accepte tout ; TaskId garantit un identifiant valide.

⚠ Les erreurs du domaine portent un code métier : la présentation choisit ensuite le message et le statut HTTP.

PARTIE 2 · APPLICATION

Les ports : de simples interfaces

```ts
// packages/application/src/ports/task-repository.ts
import type { Task, TaskId } from '@acme/domain';

export interface TaskRepository {
  save(task: Task): Promise<void>;
  byId(id: TaskId): Promise<Task | null>;
  all(): Promise<Task[]>;
}

// ports/notifier.ts
export interface Notifier {
  tacheCreee(task: Task): Promise<void>;
}

// ports/clock.ts — même le temps est injecté
export interface Clock { now(): Date; }
```

Inversion de dépendance

L'application déclare ce dont elle a besoin ; l'infrastructure s'y conforme.

Vocabulaire métier

byId, all : jamais findMany, select ou where.

Même le temps

Un port Clock rend les règles de date testables sans attendre.

PARTIE 2 · APPLICATION

Le use case orchestre

packages/application/src/task/create-task.ts

```ts
export type CreateTaskInput = { title: string; dueDate?: Date };

export class CreateTask {
  constructor(
    private readonly tasks: TaskRepository, // ports injectés
    private readonly notifier: Notifier,
  ) {}

  async execute(input: CreateTaskInput): Promise<{ id: string }> {
    const task = Task.create(input.title, input.dueDate); // règles
    await this.tasks.save(task);
    await this.notifier.tacheCreee(task);
    return { id: task.id.value };
  }
}
```

1 Reçoit un DTO

Des données simples, pas une requête HTTP.

2 Appelle le domaine

Les règles restent dans l'entité.

3 Utilise les ports

Il ignore Prisma, SMTP et Next.js.

4 Retourne un résultat

Sérialisable, prêt pour la vue.

PARTIE 2 · INFRASTRUCTURE

L’adaptateur implémente le port

packages/infra/src/prisma/prisma-task-repository.ts

```ts
import { prisma } from '@acme/db';
import type { TaskRepository } from '@acme/application';
import { TaskMapper } from './task-mapper';

export class PrismaTaskRepository implements TaskRepository {
 async save(task: Task) {
  const data = TaskMapper.toPersistence(task);
  const where = { id: data.id };
  await prisma.task.upsert({ where, create: data, update: data });
 }

 async byId(id: TaskId) {
  const row = await prisma.task.findUnique({ where: { id: id.value } });
  return row ? TaskMapper.toDomain(row) : null;
 }
}
```

Un seul endroit

Prisma n’apparaît que dans ce package.

Remplaçable

Une version en mémoire sert aux tests, une autre à la démo.

Contrat vérifié

implements TaskRepository : le compilateur contrôle la conformité.

PARTIE 2 · INFRASTRUCTURE

Mappers : deux modèles distincts

packages/infra/src/prisma/task-mapper.ts

```ts
import type { Task as TaskRow } from '@prisma/client';
import { Task, TaskId } from '@acme/domain';

export const TaskMapper = {
  toDomain(row: TaskRow): Task {
    return Task.restore(TaskId.from(row.id), {
      title: row.title, done: row.done,
      dueDate: row.dueDate ?? undefined,
    });
  },

  toPersistence(task: Task) {
    const s = task.snapshot();   // lecture contrôlée de l'état
    return { id: s.id, title: s.title, done: s.done };
  },
};
```

Pourquoi ?

Le modèle de table et le modèle métier n'évoluent pas au même rythme.

Pas de fuite

Les types @prisma/client ne sortent jamais de l'infrastructure.

Coût réel

Deux modèles à maintenir : à assumer sur un petit projet.

PARTIE 2 · ASSEMBLAGE

Injection de dépendances

apps/web/src/container/index.ts — composition root

```ts
import { PrismaTaskRepository, ResendNotifier } from '@acme/infra';
import { CreateTask, ListTasks, CompleteTask }
  from '@acme/application';

const tasks = new PrismaTaskRepository();
const notifier = new ResendNotifier(process.env.RESEND_KEY!);

export const useCases = {
  createTask: new CreateTask(tasks, notifier),
  listTasks: new ListTasks(tasks),
  completeTask: new CompleteTask(tasks),
};
```

Composition root

Un seul fichier connaît les implémentations concrètes.

Pas de framework

De simples constructeurs suffisent ; un conteneur DI reste optionnel.

En test

On remplace les ports par des fakes, sans toucher au métier.

PARTIE 2 · PRESENTATION

Next.js devient une couche mince

apps/web/src/app/tasks/page.tsx

```tsx
import { useCases } from '@/container';

export default async function TasksPage() {
  const tasks = await useCases.listTasks.execute();
  return <TaskList tasks={tasks} />;
}
```

apps/web/src/app/tasks/actions.ts

```ts
'use server';

export async function createTaskAction(
  data: FormData,
) {
  const input = CreateTaskSchema.parse({
    title: data.get('title'),
  });
  await useCases.createTask.execute(input);
  revalidatePath('/tasks');
}
```

Rôle du framework

Router, rendre, valider l'entrée, afficher les erreurs.

Ce qui disparaît

Plus de Prisma ni de règle métier dans app/.

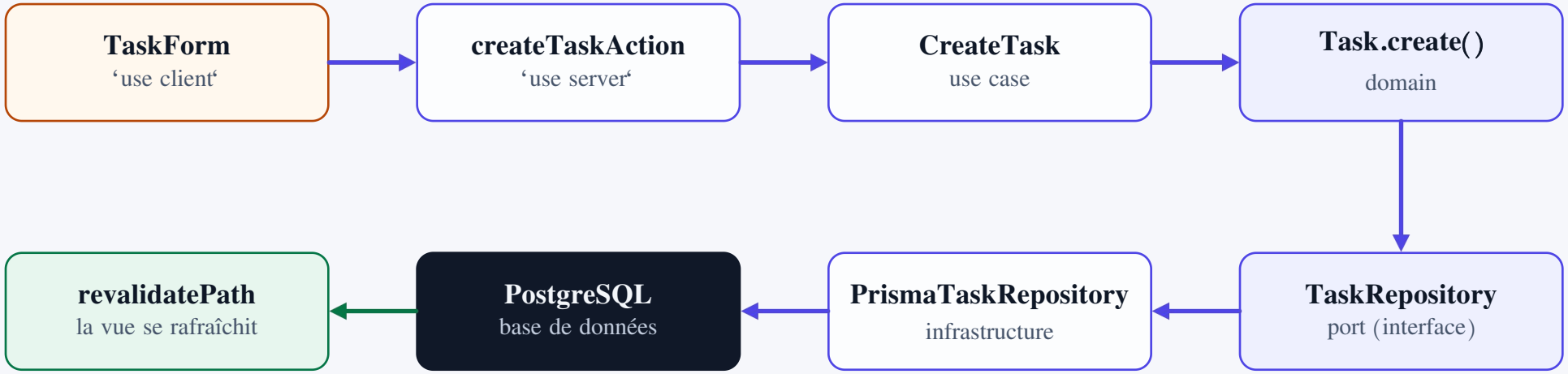
Un seul métier

Le même use case sert la page, l'action et le handler.

PARTIE 2 · SYNTHÈSE

Le flux complet d’une requête

Une requête, de l’interface à la base — et le retour



Aucune flèche ne part du domaine vers l’extérieur : il est appelé, il n’appelle personne.

- Validation à l’entrée (Zod) dans la présentation
- Le port masque Prisma au use case
- Le retour passe par revalidatePath

PARTIE 2 · ROBUSTESSE

Valider aux frontières

```ts
// apps/web/src/app/tasks/schemas.ts — frontière
import { z } from 'zod';

export const CreateTaskSchema = z.object({
  title: z.string().min(3, 'Titre trop court'),
  dueDate: z.coerce.date().optional(),
});

// dans la Server Action
const parsed = CreateTaskSchema.safeParse(brut);
if (!parsed.success) {
  return { erreurs: parsed.error.flatten().fieldErrors };
}
await useCases.createTask.execute(parsed.data);
```

Aux entrées

Formulaires, Route Handlers, variables d'environnement, webhooks.

Deux validations

Zod vérifie la forme ; le domaine vérifie les règles.

Ne pas confondre

Un schéma Zod n'est pas une entité : il ne contient aucune règle métier.

PARTIE 2 · ROBUSTESSE

Gérer les erreurs par couche

```ts
// packages/application/src/result.ts
export type Result<T> =
  | { ok: true; value: T }
  | { ok: false; code: string; message: string };

// dans la Server Action
try {
  const r = await useCases.createTask.execute(input);
  return { ok: true, value: r };
} catch (e) {
  if (e instanceof TitreTropCourt) {
    return { ok: false, code: e.code, message: e.message };
  }
  throw e;   // erreur inattendue : error.tsx + journalisation
}
```

Domaine

Lève des erreurs métier typées, porteuses d'un code.

Présentation

Traduit le code en message utilisateur et en statut HTTP.

Inattendu

Tout le reste remonte : error.tsx, journal, alerte.

PARTIE 2 · RECUL

Coûts et bénéfices réels

Bénéfices

- Règles métier testables sans base ni serveur
- Un seul endroit par règle
- ORM, framework et UI remplaçables
- Code lisible par domaine métier
- Plusieurs interfaces : web, API, cron, mobile

Coûts

- Plus de fichiers et d'indirections
- Mappers à maintenir
- Courbe d'apprentissage pour l'équipe
- Surdimensionné pour un prototype jetable
- Discipline nécessaire : sinon couches vides

Critère de décision : la durée de vie du projet et le nombre de règles métier — pas la taille du code.

03

Le monorepo

pnpm workspaces, Turborepo, packages internes, Prisma partagé et contraintes de dépendances

PARTIE 3 · CHOIX

Monorepo ou plusieurs dépôts ?

Critère	Monorepo	Plusieurs dépôts
Partager du code	Import direct	Publier un paquet npm
Changement transverse	Une seule PR	Une PR par dépôt
Versions des dépendances	Alignées	Dérive fréquente
CI	Ciblée par le graphe	Simple par dépôt
Droits d'accès	Grossiers	Fins
Poids du clone	Plus lourd	Léger

Le bon cas

Une équipe, plusieurs applications qui partagent le domaine.

Le mauvais cas

Des produits sans code commun et des équipes séparées.

Notre cas

Web, API et packages métier : le monorepo s'impose.

PARTIE 3 · STRUCTURE

Anatomie du monorepo

```
taskflow/
├── apps/
│   ├── web/          # Next.js (presentation)
│   └── admin/        # seconde app, même métier
├── packages/
│   ├── domain/       # @acme/domain
│   ├── application/  # @acme/application
│   ├── infra/        # @acme/infra
│   ├── db/           # @acme/db (Prisma)
│   ├── ui/           # @acme/ui (composants)
│   └── config/       # eslint, tsconfig, tailwind
├── pnpm-workspace.yaml
├── turbo.json
└── package.json
```

apps/

Les applications déployables : chacune a son build et son domaine d'URL.

packages/

Le code partagé : couches métier, UI, configurations.

La règle

Une app peut dépendre d'un package ; un package ne dépend jamais d'une app.

pnpm workspaces

```
# pnpm-workspace.yaml
packages:
  - 'apps/*'
  - 'packages/*'
```

Terminal

```
# dépendance interne, résolue localement
pnpm add @acme/domain --filter @acme/application

# installer tout, puis lancer une app
pnpm install
pnpm --filter web dev
```

package.json (racine)

```
{
 'name': 'taskflow',
 'private': true,
 'packageManager': 'pnpm@9.12.0',
 'scripts': {
  'dev': 'turbo run dev',
  'build': 'turbo run build',
  'test': 'turbo run test',
  'lint': 'turbo run lint'
 },
 'devDependencies': { 'turbo': '^2.1.0' }
}
```

Dans un package.json interne, la dépendance s'écrit '@acme/domain': 'workspace:*' : pnpm crée un lien, sans publier sur npm.

PARTIE 3 · OUTILLAGE

Turborepo : turbo.json

turbo.json

```
{
  “   schema“: “https://turbo.build/schema.json“,
  “tasks“: {
    “build“: {
      “dependsOn“: [“^build“],
      “outputs“: [“.next/**“, “!.next/cache/**“, “dist/**“]
    },
    “test“: { “dependsOn“: [“^build“], “outputs“: [“coverage/**“] },
    “lint“: { “dependsOn“: [“^build“] },
    “typecheck“: { “dependsOn“: [“^build“] },
    “dev“: { “cache“: false, “persistent“: true }
  }
}
```

dependsOn ^build

Le ^ signifie : d’abord le build des dépendances du package.

outputs

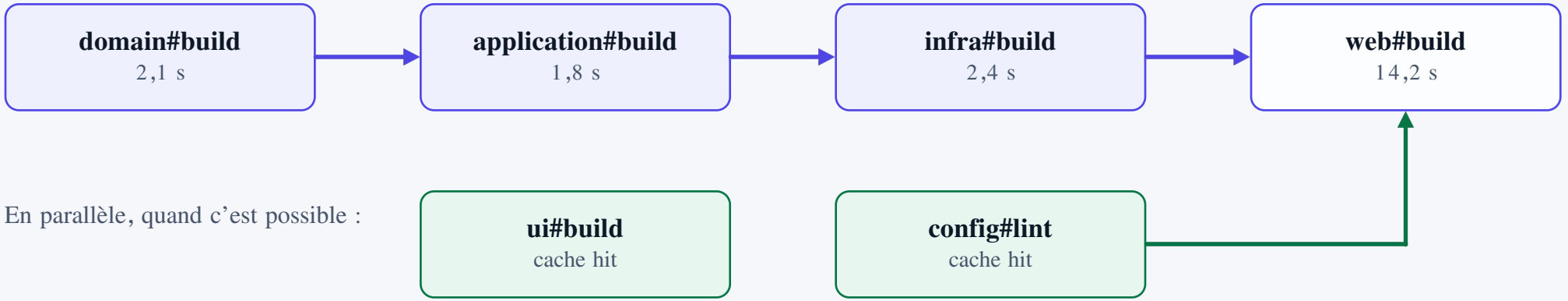
Ce que Turborepo met en cache et restaure.

cache: false

Pour dev : une tâche persistante ne se met pas en cache.

Graphe de tâches et cache

pnpm turbo run build — ordre déduit du graphe de dépendances



Deuxième exécution sans changement : 0,2 s

Cache local

Même entrée, même sortie : la tâche n'est pas rejouée.

Cache distant

Partagé entre l'équipe et la CI (Vercel ou serveur auto-hébergé).

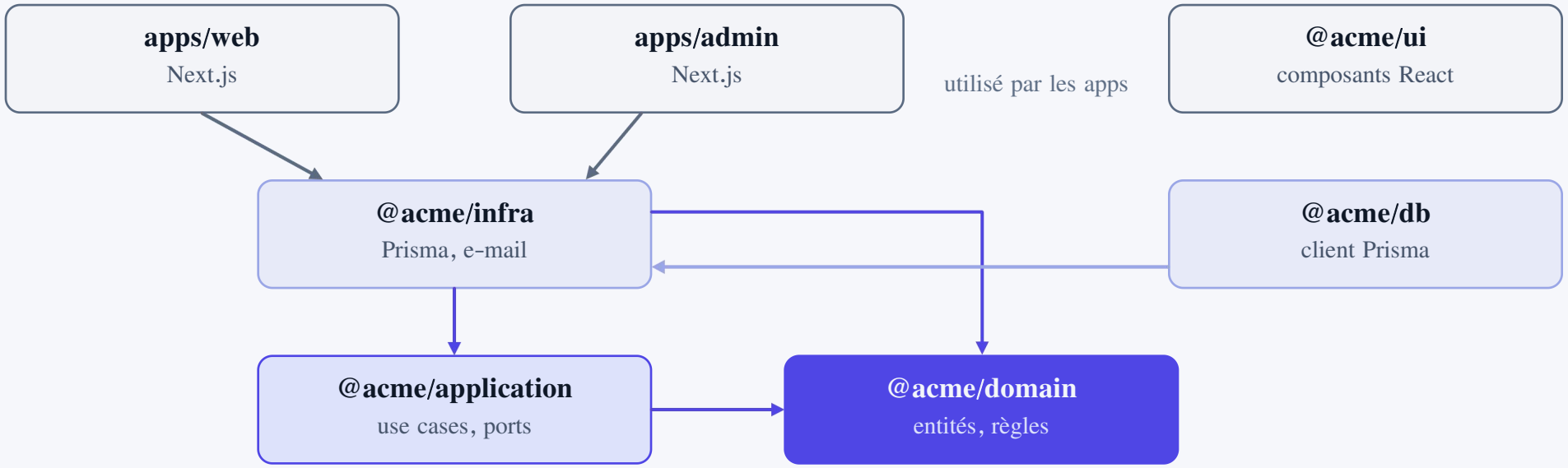
Tâches ciblées

turbo run test --filter=...[origin/main] : seulement l'impacté.

PARTIE 3 · STRUCTURE

Le graphe des packages

Les dépendances autorisées entre packages



@acme/domain ne dépend de rien

@acme/application ne connaît que le domaine

Les apps ignorent Prisma et @acme/db

Un package interne

packages/application/package.json

```
{
 ‘name‘: ‘@acme/application‘,
 ‘version‘: ‘0.0.0‘,
 ‘private‘: true,
 ‘type‘: ‘module‘,
 ‘exports‘: {
  ‘.‘: ‘./src/index.ts‘,
  ‘./ports‘: ‘./src/ports/index.ts‘
 },
 ‘dependencies‘: { ‘@acme/domain‘: ‘workspace:*‘ },
 ‘devDependencies‘: { ‘@acme/config‘: ‘workspace:*‘ },
 ‘scripts‘: { ‘typecheck‘: ‘tsc --noEmit‘, ‘test‘: ‘vitest run‘ }
}
```

Package « just-in-time »

exports pointe vers les sources : pas d’étape de build à chaque modification.

transpilePackages

Côté Next.js, déclarer les packages internes à transpiler.

Surface contrôlée

exports limite ce que les autres packages peuvent importer.

PARTIE 3 · PACKAGES

TypeScript partagé

packages/config/tsconfig/base.json

```
{
 ‘compilerOptions‘: {
  ‘target‘: ‘ES2022‘,
  ‘module‘: ‘ESNext‘,
  ‘moduleResolution‘: ‘bundler‘,
  ‘strict‘: true,
  ‘noUncheckedIndexedAccess‘: true,
  ‘skipLibCheck‘: true,
  ‘declaration‘: true
 }
}
```

apps/web/tsconfig.json

```
{
 ‘extends‘: ‘@acme/config/tsconfig/next.json‘,
 ‘compilerOptions‘: {
  ‘paths‘: { ‘@/*‘: [‘./src/*‘] }
 },
 ‘include‘: [‘src‘, ‘next-env.d.ts‘]
}
```

strict partout

Une seule base : impossible d’avoir une couche moins stricte.

Pas de chemins relatifs

On importe @acme/domain, jamais ../../../domain/src.

typecheck en tâche

Turborepo la lance package par package, en cache.

PARTIE 3 · PACKAGES

Configurations partagées

```
// packages/config/eslint/base.js
export default [
  { rules: { 'no-console': 'warn' } },
];

// packages/application/eslint.config.js
import base from '@acme/config/eslint/base';
export default base;

// packages/config/tailwind/index.ts
export const preset = { theme: { extend: { /* tokens */ } } };

// apps/web/tailwind.config.ts
import { preset } from '@acme/config/tailwind';
export default { presets: [preset], content: ['./src/**/*.tsx'] };
```

Un package config

ESLint, Prettier, TypeScript, Tailwind : une seule source de vérité.

Cohérence visuelle

Le preset Tailwind garde la même charte dans toutes les apps.

Mise à jour unique

Une règle change : tous les packages la reçoivent.

PARTIE 3 · DONNÉES

Prisma dans son propre package

```
// packages/db/prisma/schema.prisma
model Task {
  id        String    @id
  title     String
  done      Boolean   @default(false)
  dueDate   DateTime?
  createdAt DateTime  @default(now())
}
```

```
// packages/db/src/index.ts
import { PrismaClient } from '@prisma/client';

const g = globalThis as { prisma?: PrismaClient };

export const prisma = g.prisma ?? new PrismaClient();

if (process.env.NODE_ENV !== 'production') {
  g.prisma = prisma; // évite les connexions multiples
}
```

Scripts

db:generate, db:migrate, db:studio déclarés dans @acme/db.

generate avant build

Déclarer db:generate en dépendance de build dans turbo.json.

Import restreint

Seul @acme/infra importe @acme/db.

PARTIE 3 · GARDE-FOUS

Faire respecter les couches

.dependency-cruiser.cjs — l'architecture devient un test

```
module.exports = {
 forbidden: [
  {
   name: 'domain-pur',
   severity: 'error',
   from: { path: '^packages/domain' },
   to: { path: 'node_modules/(@prisma|next|react)' },
  },
  {
   name: 'presentation-sans-orm',
   severity: 'error',
   from: { path: '^apps/web' },
   to: { path: '^packages/(db|infra)/' },
  },
 ],
};
```

En CI

pnpm depcruise échoue : la pull request est bloquée.

Alternatives

eslint-plugin-boundaries, ou les tags de modules avec Nx.

Pourquoi ?

Une règle écrite dans un document finit par être oubliée.

PARTIE 3 · GARDE-FOUS

Configuration typée

```
// packages/config/src/env.ts
import { z } from 'zod';

const schema = z.object({
  DATABASE_URL: z.string().url(),
  RESEND_KEY: z.string().min(1),
  NODE_ENV: z.enum(['development', 'test', 'production']),
});

export const env = schema.parse(process.env);   // échoue au démarrage

// turbo.json : déclarer les variables qui invalident le cache
// 'build': { 'env': ['DATABASE_URL', 'RESEND_KEY'] }
```

Échec rapide

Une variable manquante arrête le démarrage, pas la requête client.

Cache correct

Les variables déclarées dans turbo.json entrent dans la clé de cache.

Côté client

Seules les variables NEXT_PUBLIC_ sont exposées au navigateur.

PARTIE 3 · LIVRAISON

Versions et journal des changements

Terminal

```
# 1. Décrire le changement (dans la PR)
pnpm changeset
# → quels packages ? patch / minor / major ? résumé ?

# 2. Appliquer les versions (CI ou local)
pnpm changeset version

# 3. Publier (si des packages sont publics)
pnpm publish -r
```

CHANGELOG automatique

Chaque changeset devient une entrée lisible par l'équipe.

Packages privés

Même sans publication npm, le journal documente les évolutions.

Discipline

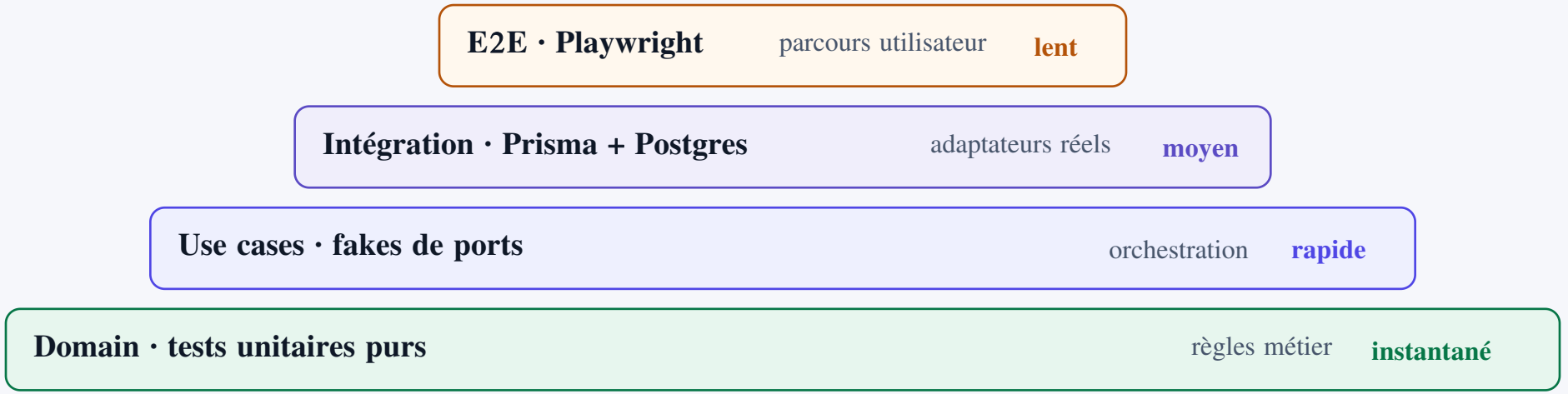
Pas de changeset, pas de merge : la règle est dans la CI.

04

Qualité et livraison

Stratégie de tests par couche, intégration continue, déploiement et mise en production

Une stratégie par couche



- ✓ Beaucoup de tests de domaine : ils sont gratuits
- ✓ Peu de tests E2E : les parcours critiques seulement
- ✓ Un test par couche, pas un test par fichier

PARTIE 4 · TESTS

Tester un use case sans base

packages/application/src/task/create-task.test.ts

```ts
class FakeTaskRepository implements TaskRepository {
  readonly items: Task[] = [];
  async save(t: Task) { this.items.push(t); }
  async byId() { return null; }
  async all() { return this.items; }
}

it('crée une tâche et notifie', async () => {
  const repo = new FakeTaskRepository();
  const notifier = { tacheCreee: vi.fn() };
  const useCase = new CreateTask(repo, notifier);
  await useCase.execute({ title: 'Réviser' });
  expect(repo.items).toHaveLength(1);
  expect(notifier.tacheCreee).toHaveBeenCalledOnce();
});
```

Millisecondes

Aucune base, aucun serveur, aucun réseau.

Fakes

Une implémentation en mémoire du port, réutilisable partout.

Ce que l'on teste

Le comportement attendu, pas l'implémentation.

PARTIE 4 · TESTS

Tester l’adaptateur pour de vrai

packages/infra/src/prisma/prisma-task-repository.test.ts

```ts
beforeAll(async () => {
  container = await new PostgreSqlContainer().start();
  process.env.DATABASE_URL = container.getConnectionUri();
  execSync('pnpm --filter @acme/db db:migrate deploy');
});

it('enregistre puis relit une tâche', async () => {
  const repo = new PrismaTaskRepository();
  const task = Task.create('Réviser les ports');
  await repo.save(task);
  const relue = await repo.byId(task.id);
  expect(relue?.snapshot().title).toBe('Réviser les ports');
});
```

Vraie base

Testcontainers démarre un PostgreSQL jetable.

Ce que l’on vérifie

Le mapping, les requêtes, les contraintes — pas le métier.

Volume

Quelques tests par repository suffisent : ils sont lents.

PARTIE 4 · TESTS

Les parcours critiques en E2E

```ts
// apps/web/e2e/tasks.spec.ts
import { test, expect } from '@playwright/test';

test('un utilisateur ajoute une tâche', async ({ page }) => {
  await page.goto('/tasks');
  await page.getByLabel('Titre').fill('Préparer le TP');
  await page.getByRole('button', { name: 'Ajouter' }).click();

  await expect(page.getByText('Préparer le TP')).toBeVisible();
});
```

Point de vue utilisateur

Le test clique comme un humain : aucun détail interne.

Sélecteurs accessibles

getByRole et getByLabel : robustes et bons pour l'accessibilité.

À limiter

Trois à dix parcours : connexion, création, paiement…

PARTIE 4 · LIVRAISON

Intégration continue

.github/workflows/ci.yml

```yaml
name: ci
on: [push, pull_request]
jobs:
  verify:
    runs-on: ubuntu-latest
    env:
      TURBO_TOKEN:    {{ secrets.TURBO_TOKEN }}
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm turbo run lint typecheck test build
```

Une seule commande

Turborepo ordonne lint, typecheck, test et build.

Cache distant

Les tâches inchangées sont restaurées, pas rejouées.

fetch-depth: 0

Nécessaire pour comparer avec la branche principale et cibler.

Lockfile figé

--frozen-lockfile : la CI échoue si le lockfile n'est pas à jour.

PARTIE 4 · LIVRAISON

Déployer : Vercel ou Docker

Dockerfile (extrait)

```
# next.config.ts : output: 'standalone'
FROM node:22-alpine AS base
WORKDIR /app
COPY . .
RUN corepack enable && pnpm install --frozen-lockfile
RUN pnpm turbo run build --filter=web

FROM node:22-alpine AS runner
WORKDIR /app
COPY --from=base /app/apps/web/.next/standalone ./
COPY --from=base /app/apps/web/.next/static ./apps/web/.next/static
CMD ["node", "apps/web/server.js"]
```

Vercel

Root directory = apps/web ; le cache Turborepo est intégré.

Docker

output standalone produit une image légère pour un VPS ou Kubernetes.

Migrations

prisma migrate deploy dans une étape séparée du déploiement.

PARTIE 4 · EXPLOITATION

Observer en production

Journalisation structurée

Des logs JSON avec un identifiant de requête, émis depuis l'infrastructure.

Traçage

OpenTelemetry : suivre une requête de la page jusqu'à la base.

Suivi des erreurs

Sentry ou équivalent : erreurs serveur et client, avec la version déployée.

Core Web Vitals

LCP, INP, CLS : mesurés sur le trafic réel, pas en local.

Les ports aident aussi ici : un Logger injecté laisse le domaine lisible, sans console.log dispersés.

PARTIE 4 · EXPLOITATION

Checklist de mise en production

- ☑ Variables d'environnement validées au démarrage
- ☑ Lint, typecheck, tests et build verts en CI
- ☑ Server Actions : validation et autorisation
- ☑ Journalisation et suivi des erreurs branchés
- ☑ Plan de retour arrière (rollback) documenté
- ☑ Migrations appliquées dans une étape dédiée
- ☑ Contraintes de dépendances vérifiées (depcruise)
- ☑ Pages critiques : stratégie de rendu choisie
- ☑ Sauvegardes de base de données testées
- ☑ Budget de performance surveillé (Web Vitals)

PARTIE 4 · PROJET FIL ROUGE

Projet : TaskFlow

```
taskflow/
├── apps/web/src/
│   ├── app/              # routes, layouts, actions
│   ├── container/        # composition root
│   └── components/
├── packages/
│   ├── domain/src/task/    # Task, TaskId, erreurs
│   ├── application/src/    # use cases + ports
│   │   ├── task/
│   │   └── ports/
│   ├── infra/src/prisma/   # repositories + mappers
│   ├── db/prisma/          # schema + migrations
│   ├── ui/src/             # composants partagés
│   └── config/             # eslint, tsconfig, tailwind
└── turbo.json
```

Périmètre attendu

- ✓ Créer, terminer et supprimer une tâche
- ✓ Filtrer : toutes, actives, urgentes
- ✓ Règle métier : urgence calculée dans le domaine
- ✓ Deux adaptateurs : Prisma et en mémoire
- ✓ Tests : domaine, use cases, un parcours E2E
- ✓ CI verte avec contraintes de dépendances

> Livrable : le dépôt, plus un fichier ARCHITECTURE.md qui justifie les choix.

PARTIE 4 · PROJET FIL ROUGE

Étapes de réalisation

1 Squelette

pnpm workspaces, turbo.json, package config partagé.

2 Domaine

Task, TaskId, erreurs métier et leurs tests unitaires.

3 Use cases

CreateTask, ListTasks, CompleteTask et les ports.

4 Fakes

Repository en mémoire et tests des use cases.

5 Prisma

Package db, schéma, repository et mappers.

6 Interface

App Router, Server Components, Server Actions, Zod.

7 Garde-fous

dependency-cruiser, env typé, CI GitHub Actions.

8 Livraison

Un parcours E2E, puis déploiement Vercel ou Docker.

PARTIE 4 · RECUL

Pièges les plus fréquents

⚠ Couches vides

Un use case qui appelle directement le repository sans aucune règle : le domaine devient décoratif.

⚠ Entité anémique

Une classe avec seulement des getters et setters : les règles repartent dans les composants.

⚠ Fuite de Prisma

Un type @prisma/client qui remonte jusqu'au composant : tout le découplage est perdu.

⚠ API interne inutile

Des Route Handlers appelés par ses propres pages : un aller-retour HTTP pour rien.

⚠ « use client » en haut

La page entière devient cliente pour un seul bouton : le bénéfice des RSC disparaît.

⚠ Barrel files géants

Un index.ts qui réexporte tout : imports circulaires et cache Turborepo inefficace.

SYNTHÈSE

Récapitulatif

app/
Routes, layouts, actions : couche mince

domain
Entités et règles, sans dépendance

application
Use cases et ports

infra
Prisma, e-mail : adaptateurs

container
Un seul point d'assemblage

turbo.json
Graphe de tâches et cache

depcruise
Les couches vérifiées en CI

vitest
Tests rapides par couche

métier | framework

Next.js, Prisma et Turborepo sont des outils remplaçables ; vos règles métier, non.

POUR ALLER PLUS LOIN

Exercices et ressources

1. **Second adaptateur**
   Écrire un InMemoryTaskRepository et faire tourner l'app sans base.

2. **Nouveau use case**
   ArchiverTachesTerminees, avec sa règle dans le domaine et ses tests.

3. **Deuxième application**
   Créer apps/admin qui réutilise les mêmes use cases.

4. **Lint d'architecture**
   Ajouter une règle depcruise, puis provoquer volontairement son échec.

Ressources

nextjs.org/docs
Documentation officielle de Next.js

turborepo.com/docs
Guides monorepo et cache distant

pnpm.io/workspaces
Workspaces et protocole workspace:

prisma.io/docs
Schéma, migrations, bonnes pratiques

blog.cleancoder.com
L'article d'origine sur la Clean Architecture



Questions ?

Place au projet : commencez par le domaine, pas par la page.

Med Amine Ben Rhouma · Next.js, Clean Architecture et Monorepo